Technical Journal Entry: Resource Storage & Seed Bank

- **Module:** Resource Storage and Seed Bank Initialization (Plant class + UI unlock)
- **Time/Space Complexity:** O(1) for Plant object instantiation; O(k) time/space for populating the seed dropdown, where k is the number of plant tuples passed to `addPlantsToDropdown`.
- **Architectural Choice:** Used a simple class (`Plant`) to model garden resource entities with encapsulated state (`name`, `water_need`, `health`), then called two bridge functions (`updateResources`, `addPlantsToDropdown`) to sync Python state with the JS/DOM layer and unlock the Resources panel.
- **AI Reflection Note:**
    - **The Prompt Used (RTCC):**
      "Act as an expert Python developer. Task: Write a Plant class and initialize the garden's resource system and seed dropdown. Context: Python 3.11+, vanilla Python compatible with browser Pyodide execution, must call exact JS bridge functions `updateResources` and `addPlantsToDropdown`. Constraints: Must include a `__init__` storing name, water_need, and health=100, print a creation message, create at least one Plant object, call `updateResources` with exact specified values, call `addPlantsToDropdown` with at least 3 tuples, and end with an exact success print statement."
    - **What the AI Initially Got Wrong:**
      N/A — output matched requirements on first generation (no missing fields, correct function signatures, correct tuple format).
    - **How I Fixed It:**
      N/A — no correction cycle was needed for this module.
- **Screenshots:** *[insert screenshot of unlocked Resources panel + console output here]*

Technical Prompt Log

Module: Sanctuary Garden Grid (2D Array) — Topic 4 Implementation

Session Context: Diagnosing and completing the garden grid feature in the Python-Based Garden Simulator (DSA finals project), from initial bug discovery through full implementation.

Entry 1 — Diagnosing the Locked Grid

- **Prompt Used:** "clicking on the interactive summary reveals a blurred grid. what is this?"
- **What the AI Found:** The `view-game` section carries a CSS `.locked` class (blur + disabled pointer events) that only clears when `unlockFeature("grid")` runs, which is gated behind a Topic 4 code submission — but `topicTemplates` only defined Topics 1–3, so no UI path existed to trigger it.
- **Outcome:** Root cause identified; no fix applied yet.

Entry 2 — Requesting Workarounds

- **Prompt Used:** "give me workarounds to unlock this. since there is no 4th topic."
- **What the AI Provided:** Four options — console call to `unlockFeature("grid")`, running the same call from the Python editor (since it's registered as a Pyodide global), a `javascript:` bookmarklet, and the proper fix of adding a real Topic 4 entry.
- **Outcome:** User applied the console/editor workaround to confirm the unlock mechanism worked.

Entry 3 — Surfacing the Deeper Bug

- **Prompt Used:** "did this and it unlocked but it's not functionable"
- **What the AI Initially Got Wrong (pre-existing code, not AI-introduced):** `renderGarden()` never attached any `onclick` handler to grid cells, and `gardenPlants` was a flat object indexed by a single position rather than a 2D structure — so unlocking only removed the visual overlay, not the missing interactivity.
- **Outcome:** Problem scoped as two separate issues: (1) no Topic 4 exists, (2) grid has no click logic or real 2D structure.

Entry 4 — Full Implementation (RTCC)

- **Prompt Used:** "'2D list and array' is a requirement for this project... follow the structure and function of the 3 base topics... create a 4th topic and make the existing grid

functioning properly... do not introduce unauthorized external libraries... [list: Tailwind CSS, Lucide Icons, Pyodide, Vanilla JS/HTML5 DOM]... respond in a code snippet that highlights every single change."

- **What the AI Initially Got Wrong:** N/A on this pass — the AI directly refactored `gardenPlants` into a true `[row][col]` 2D array, added `initGardenGrid(rows, cols)`, rewrote `addPlantToGrid(row, col, name)`, added real click handlers in `renderGarden()`, gated the Topic 4 unlock on `code.includes("initGardenGrid")` (matching Topics 1–3's pattern), and added the Topic 4 template using a Python 2D list comprehension.
- **How It Was Verified:** Each change was isolated into its own labeled diff (function/section named as its "location" in source), followed by the complete revised file delivered as a downloadable artifact.

Entry 5 — Requesting Student-Facing Input Code

- **Prompt Used:** "generate input code to put in the student code editor for topic 4"
- **What the AI Provided:** The exact starter snippet (`garden_grid = [[None for _ in range(cols)] for _ in range(rows)]` → `initGardenGrid(rows, cols)`) formatted for direct paste into the editor, with a short explanation of why it satisfies both the 2D-array requirement and the unlock condition.

Entry 6 — Formalizing the Reusable Prompt Pattern

- **Prompt Used:** "generate prompt to receive this exact format of code snippets and the changes made. add a location on where the source code is the changes made. after that, an additional file for the revised version."
- **Outcome:** A reusable prompt template was produced, codifying: numbered per-change diff blocks with named source locations, followed by a separately delivered full revised file rather than inline pasting.

Architectural Summary: The Topic 4 fix mirrors the existing Topics 1–3 pattern exactly — student writes code in the editor, runs it, `runPythonCode()` checks for a required function call in the submitted code, and only then unlocks the UI panel via `unlockFeature()`. The grid's `gardenPlants` array is now a genuine 2D list (`rows × cols`), satisfying the DSA 2D-array requirement on both the Python and JavaScript sides, with click handlers wired directly to `(row, col)` indices rather than a flat offset.

Technical Journal Entry:

- **Module:** Skill Tree (Strict Binary Search Tree with BFS, DFS and Node Mutation)
- **Time/Space Complexity:**
    - BST insert and search: O(h) time, which is O(log n) for a balanced tree and O(n) worst case.
    - BFS layout and tier scaling: O(n) time, O(w) extra space for the queue, where w is the widest level.
    - DFS cascading refund and branch value: O(n) time, O(h) space for the recursion or stack.
    - Node mutation (buy or refund of one node): O(1).
    - O(n) space overall for storing n skill nodes.
- **Architectural Choice:** I used a strict Binary Search Tree where every skill has a unique `Skill_ID`, a smaller ID goes left and a larger ID goes right. Each skill has exactly one prerequisite, its parent, so converging dependencies are not possible. BFS (a queue) walks the tree level by level to assign each node a row, a column and a depth tier, and the tier drives the cost formula `Base Cost × 1.5^tier`. DFS (post-order) refunds a skill by first refunding everything in its left and right subtrees, so no unlocked skill is ever left without its parent. A stack-based DFS also sums the coins and hope invested in a branch. Buying or refunding mutates the node in place (`is_unlocked`, `current_rank`). That triggers a recalculation of global stats such as water capacity and harvest yield, and the UI redraws the node.
- **AI Reflection Note:**
    - **The Prompt Used (RTCC):**
      "Act as an expert Python and JavaScript developer. Task: Implement a skill tree for my garden simulator using a strict Binary Search Tree. Use BFS for level-order layout (row and column) and tier cost scaling (Base Cost × 1.5^tier). Use DFS for cascading refunds and branch value calculation. Use node mutation (`is_unlocked`, `current_rank`) to update game stats and the UI. Context: single HTML file (the provided Week 3 code), Python 3.11+ vanilla Python running in browser Pyodide, existing Topics 1–5 must keep working. Constraints: no converging dependencies, a maximum of two children per node, each skill has one parent prerequisite, and nothing crashes on invalid actions."
    - **What the AI Initially Got Wrong:**
        1. **Layout:** The first version of the tree UI clipped the leftmost and rightmost leaf nodes at the edge of the panel, and the connecting lines ran through the skill name labels.
        2. **Code injection:** The first attempt to inject the Python code into the JavaScript template literal failed because a stray backtick in a Python docstring would have broken the page. A safety check caught it before it shipped.
        3. **Tests:** Two failing checks in the first browser test run came from the test itself, not the game. One was a case-sensitive text match against

uppercase CSS text, and the other was a console error from a blocked font request.

- **How I Fixed It:**
    1. I added padding with an inner container so edge nodes fit, gave labels a solid background so lines pass behind them, and tuned node sizes so the tree fits without horizontal scrolling on desktop.
    2. I removed the backtick from the docstring and added an assertion that rejects backticks, backslashes and `${` in the injected code.
    3. I made the text match case-insensitive and ignored the blocked-resource console message. The test run then passed 38/38 checks in a real headless browser with Pyodide.

- **Screenshots:**

Python-Based Garden Simulator

SDG 2, 13, 15

DAY 1 | SCORE 0%

Resource Storage

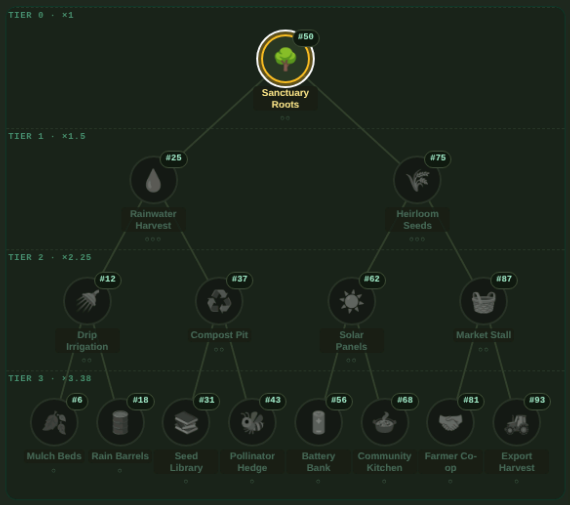
- Water Reserves: 200 / 200 L
- Seed Inventory: 90 / 100
- Solar Energy: 110 / 120 Wh
- Community Hope: 80 / 100
- Coins / Dues: 999 Coins

Interactive Sanctuary | Skill Tree | Student Code Editor

Sanctuary Skill Tree

Strict BST: **left = smaller Skill_ID**, **right = larger**. One prerequisite per skill (its parent).

COINS 999 | UNLOCKED 0 / 15



BFS level order: #50 → #25 → #75 → #12 → #37 → #62 → #87 → #6 → #18 → #31 → #43 → #56 → #68 → #81 → #93

Last DFS refund (post-order): no refund yet

Sanctuary Roots — Skill_ID 50

+10 Seed capacity per rank

AVAILABLE | Rank 0 / 2

Prerequisite: **none (root)**

Children: **◀ #25 Rainwater Harvest · ▶ #75 Heirloom Seeds**

BFS tier cost: **8 × 1.5⁰ = 8 coins / rank**

DFS branch value: **0 coins in 0 unlocked skills**

Unlock · 8 coins | Nothing to refund

Refunding a skill makes DFS dive into its left and right subtrees and refund every unlocked descendant bottom-up.

◀ LEFT BRANCH OF ROOT
#25 Rainwater Harvest
0 coins +0 hope
0 unlocked skills · summed by DFS

▶ RIGHT BRANCH OF ROOT
#75 Heirloom Seeds
0 coins +0 hope
0 unlocked skills · summed by DFS

GLOBAL STATS (RECALCULATED AFTER EVERY NODE MUTATION)

- 💧 Water capacity: 200 → 200 L — no bonus yet
- 🌱 Seed capacity: 100 → 100 — no bonus yet
- ☀️ Solar capacity: 120 → 120 Wh — no bonus yet
- 💗 Community Hope: +0 — bar 80 / 100
- 🌾 Harvest yield: +0% — harvest = 5 coins

Climate Queue (FIFO)

- Typhoon
- Drought

Action History (Stack)

- → Skill Tree Initialized (BST + BFS + DFS)
- → Growth Stage Hierarchy Initialized (Tree)
- → Sanctuary Garden Grid Initialized (2D Array)
- → planted at [1] [1]
- → planted at [1] [2]
- → Resources & Seed Bag Initialized

Skill Tree unlocked: open the Skill Tree tab

Undo (Stack Pop)

Growth Hierarchy (Tree)

- 🌰 ↓ child node
- 🌱 ↓ child node
- 🌳 ↻ loops back to root

Click a planted tile to cycle it through the tree.

Python-Based Garden Simulator

SDG 2, 13, 15

DAY 1 | SCORE 0%

Resource Storage

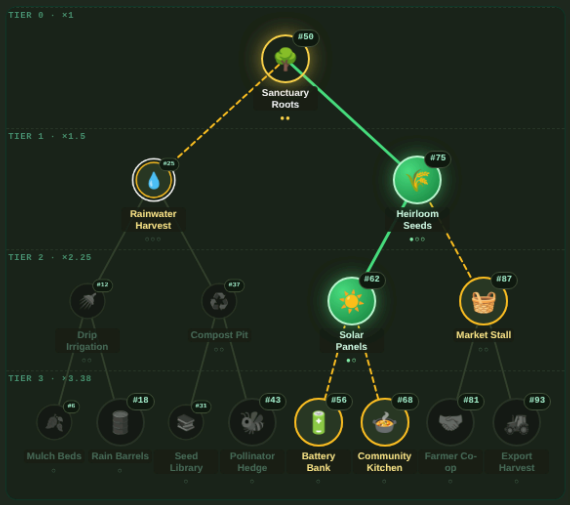
- Water Reserves: 200 / 200 L
- Seed Inventory: 90 / 120
- Solar Energy: 110 / 140 Wh
- Community Hope: 80 / 100
- Coins / Dues: 935 Coins

Interactive Sanctuary | Skill Tree | Student Code Editor

Sanctuary Skill Tree

Strict BST: **left = smaller Skill_ID**, **right = larger**. One prerequisite per skill (its parent).

COINS 935 | UNLOCKED 3 / 15



BFS level order: #50 → #25 → #75 → #12 → #37 → #62 → #87 → #6 → #18 → #31 → #43 → #56 → #68 → #81 → #93

Last DFS refund (post-order): #6 → #12 → #31 → #37 → #25

Rainwater Harvest — Skill_ID 25

+25 L Water capacity per rank

AVAILABLE

Rank 0 / 3

Prerequisite: **#50 Sanctuary Roots ✓**

Children: **◀ #12 Drip Irrigation · ▶ #37 C… Pit**

BFS tier cost: **8 × 1.5¹ = 12 coins / rank**

DFS branch value: **0 coins in 0 unlocked skills**

Unlock · 12 coins | Nothing to refund

Refunding a skill makes DFS dive into its left and right subtrees and refund every unlocked descendant bottom-up.

◀ LEFT BRANCH OF ROOT
#25 Rainwater Harvest
0 coins +0 hope
0 unlocked skills · summed by DFS

▶ RIGHT BRANCH OF ROOT
#75 Heirloom Seeds
48 coins +0 hope
2 unlocked skills · summed by DFS

GLOBAL STATS (RECALCULATED AFTER EVERY NODE MUTATION)

- Water capacity: 200 → 200 L — no bonus yet
- Seed capacity: 100 → 120 — +20 from skills
- Solar capacity: 120 → 140 Wh — +20 Wh from skills
- Community Hope: +0 — bar 80 / 100
- Harvest yield: +10% — harvest = 6 coins

Climate Queue (FIFO)

- Typhoon
- Drought

Action History (Stack)

- → Refunded Rainwater Harvest branch: 5 skill(s), +143 coins
- → Unlocked Solar Panels (rank 1/2) for 27 coins
- → Unlocked Heirloom Seeds (rank 1/3) for 21 coins
- → Unlocked Seed Library (rank 1/1) for 30 coins
- → Unlocked Compost Pit (rank 1/2) for 27 coins
- → Unlocked Mulch Beds (rank 1/1) for 27 coins
- → Unlocked Drip Irrigation (rank 1/2) for 23 coins
- → Unlocked Rainwater Harvest (rank 3/3) for 12 coins

Refunded Rainwater Harvest branch: 5 skill(s), +143 coins

Undo (Stack Pop)

Growth Hierarchy (Tree)

- ↓ child node
- ↓ child node
- ↻ loops back to root

Click a planted tile to cycle it through the tree.